This code shows how to combine retrieval, prompt engineering, and language model generation using the LangChain.js chain system—a system that creates a lot of the nuances for the developer.

The provided `performRAG` method creates a chat prompt template that instructs the language model to answer questions based solely on the provided context. It then uses the retriever to fetch and format relevant documents according to the input query, accepts the question as input, and generates a response using an LLM. The function invokes this processing chain to generate the final response while also retrieving the source documents used for reference. Ultimately, it returns an object containing both the generated answer and the corresponding source documents.

With all the essential components of a RAG system in place, the next step is to create a user interface that allows users to perform queries based on the provided documents.

7.2.5 Web app demonstration

The web application uses our existing UI components with a key enhancement: integrating a RAG system. The core functionality remains unchanged, but we've added a sophisticated document retrieval and context-aware response generation mechanism.

RAG INDEX CONFIGURATION

To enable the RAG functionality, you must specify the absolute path to your vector index. This is done by setting the `RAG_INDEX_PATH` environment variable in the .env file:

```
RAG_INDEX_PATH=/absolute/path/to/your/rag_index
```

If you run the `indexDocuments.js` script inside the scripts folder, it will create a folder in the project directory that contains the embeddings. Provide the absolute path of this folder for the application to load them for retrieval.

KEY CODE CHANGES

The following listing shows the important code snippet that initializes the RAG system in the actions.jsx file.

Listing 7.10 Initializing the RAG system on the server

```
import { RAG } from '@/lib/RAG';
import { getAbsoluteRAGIndexPath } from '@/lib/utils';
export async function initializeRAG() {
  if (!ragSystem) {
    ragSystem = new RAG(apiKey);
    const indexPath = getAbsoluteRAGIndexPath();
    await ragSystem.loadIndex(indexPath);
  }
  return ragSystem;
}

export async function continueConversation(input) {
  const aiState = getMutableAIState();

  try {
```

Imports the RAG class, which encapsulates the RAG system functionality

Defines an async function to initialize the RAG system